Métodos basados en árboles
Taller 4 - Módulo 5 DCD

XBG

Tabla de Contenidos

1 Informe Científico **4**
- 1.1 Importación de datos . . . 4
- 1.2 Análisis Exploratorio de Datos . . . 5
  - 1.2.1 Gráficos . . . 5
- 1.3 Particionamiento de datos . . . 7
- 1.4 Definición genérica de modelo y ajuste . . . 7
- 1.5 Predicción con datos test . . . 9
  - 1.5.1 Cálculo de matriz de confusión . . . 9
  - 1.5.2 Rendimiento . . . 10
- 1.6 Revisión de hiperparámetros "Cost Complexity" para la poda (pruning) del árbol . . . 11

2 Bagging **12**
- 2.1 Cálculo del rendimiento en testing . . . 13

3 Random Forest **15**

4 Comparación de modelos usando ROC **17**

Referencias Bibliográficas **20**

Listado de Figuras

1.1 default por duración del préstamo 5
1.2 default por número de residencias 6
1.3 default por edad del cliente 7
1.4 Diagrama de árbol de clasificación 9
1.5 Matriz de confusión 10

1 Informe Científico

```
pacman::p_load(tidyverse, #para manip.datos
               tidymodels,
               ISLR,
               janitor,
               inspectdf,
               GGally,
               discrim,
               randomForest,
               rpart.plot,
               kableExtra,
               pROC)

theme_set(theme_minimal())
```

1.1 Importación de datos

```
datos <- read_delim("credit.csv")
```

```
inspect_types(datos)
```

```
# A tibble: 2 x 4
  type        cnt  pcnt col_name
  <chr>     <int> <dbl> <named list>
1 character    13  61.9 <chr [13]>
2 numeric       8  38.1 <chr [8]>
```

```
datos$default <- factor(datos$default)
levels(datos$default)
```

```
[1] "1" "2"
```

Tabla 1.1: Distribución de clases de la variable `default`

datos$default	n	percent
1	700	0.7
2	300	0.3

1.2 Análisis Exploratorio de Datos

```
res_imp <- tabyl(datos$default)

kbl(res_imp, booktabs = T) |>
  kable_styling(latex_options = c("hold_position"))
```

1.2.1 Gráficos

```
ggplot(datos, aes(x = default, y = months_loan_duration)) +
  geom_boxplot()
```

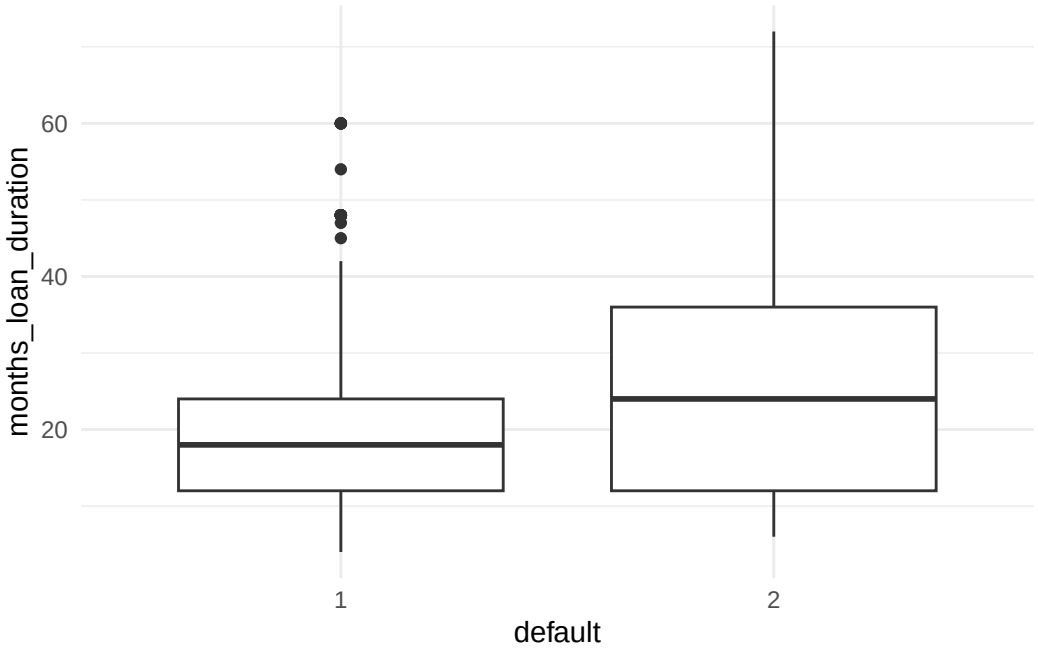


Figura 1.1: default por duración del préstamo

```
ggplot(datos) +
  geom_bar(aes(x = months_loan_duration, fill = default), position = "fill") +
  facet_wrap(~residence_history)
```

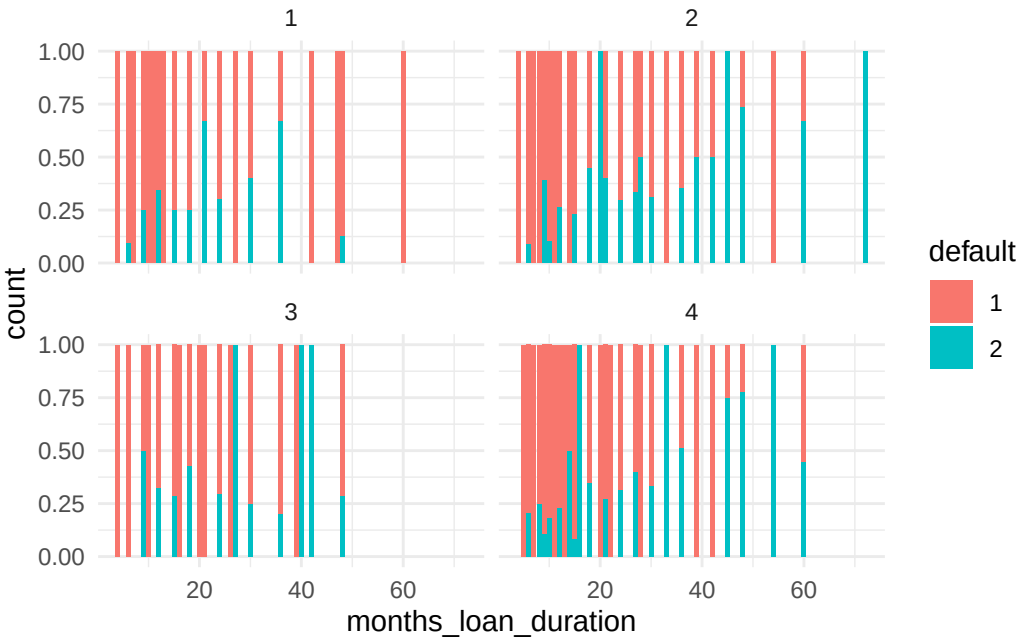


Figura 1.2: default por número de residencias

```
ggplot(datos, aes(x = default, y = age)) +
  geom_boxplot()
```

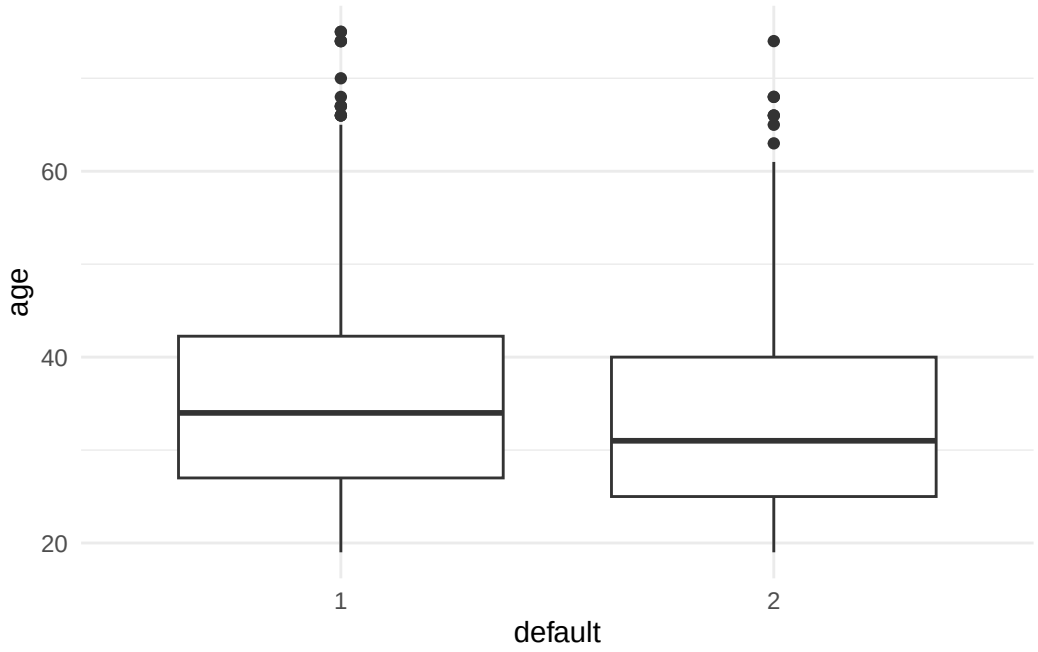


Figura 1.3: default por edad del cliente

1.3 Particionamiento de datos

```
datos <- datos |>
  dplyr::select(age, months_loan_duration, amount, default)

set.seed(100)
particion <- initial_split(datos, prop = 0.70, strata = default)

datos_entre <- training(particion)
datos_test <- testing(particion)
```

1.4 Definición genérica de modelo y ajuste

```
tree_spec <- decision_tree() |>
  set_engine("rpart") |>
  set_mode("classification") |>
  fit(default ~., data= datos_entre)

tree_spec
```

```
parsnip model object

n= 699

node), split, n, loss, yval, (yprob)
      * denotes terminal node

  1) root 699 210 1 (0.6995708 0.3004292)
    2) months_loan_duration< 15.5 312  63 1 (0.7980769 0.2019231) *
    3) months_loan_duration>=15.5 387 147 1 (0.6201550 0.3798450)
      6) age>=24.5 339 117 1 (0.6548673 0.3451327)
       12) months_loan_duration< 34.5 231  69 1 (0.7012987 0.2987013)
         24) amount>=1247 207  56 1 (0.7294686 0.2705314) *
         25) amount< 1247 24  11 2 (0.4583333 0.5416667)
           50) amount< 1178.5 16   6 1 (0.6250000 0.3750000) *
           51) amount>=1178.5 8   1 2 (0.1250000 0.8750000) *
       13) months_loan_duration>=34.5 108  48 1 (0.5555556 0.4444444)
         26) age>=26.5 99  40 1 (0.5959596 0.4040404)
           52) amount>=3069.5 82  29 1 (0.6463415 0.3536585)
            104) age< 41.5 59  17 1 (0.7118644 0.2881356) *
            105) age>=41.5 23  11 2 (0.4782609 0.5217391)
              210) amount>=8461.5 10   3 1 (0.7000000 0.3000000) *
              211) amount< 8461.5 13   4 2 (0.3076923 0.6923077) *
           53) amount< 3069.5 17   6 2 (0.3529412 0.6470588) *
         27) age< 26.5 9   1 2 (0.1111111 0.8888889) *
      7) age< 24.5 48  18 2 (0.3750000 0.6250000) *
```

```
sum(tree_spec$fit$frame$var=="<leaf>")
```

```
[1] 10
```

```
rpart.plot(tree_spec$fit, roundint = FALSE)
```

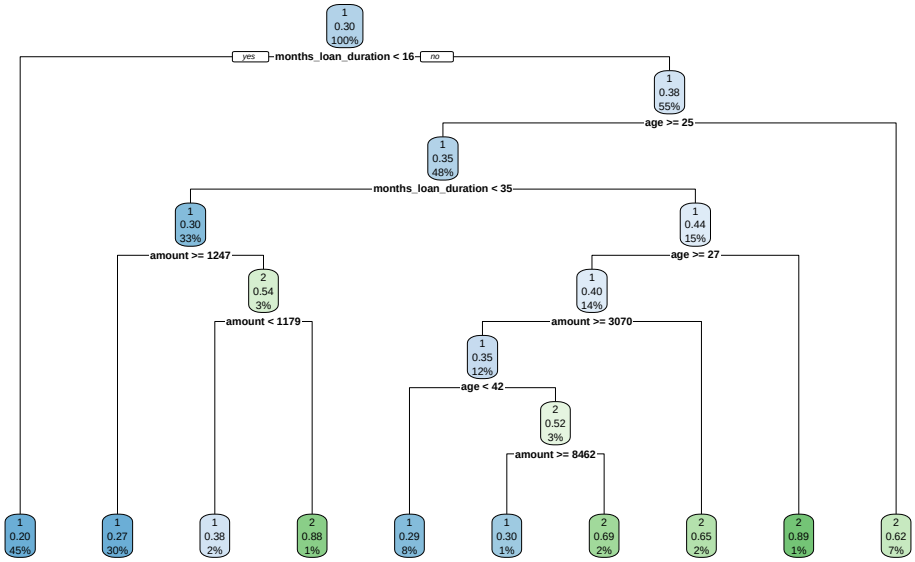


Figura 1.4: Diagrama de árbol de clasificación

1.5 Predicción con datos test

```
prest_treepred <- augment(tree_spec, new_data = datos_test, type = "class")
```

1.5.1 Cálculo de matriz de confusión

```
prest_treepred |>
  conf_mat(truth = default, estimate = .pred_class) |>
  autoplot(type="heatmap")
```

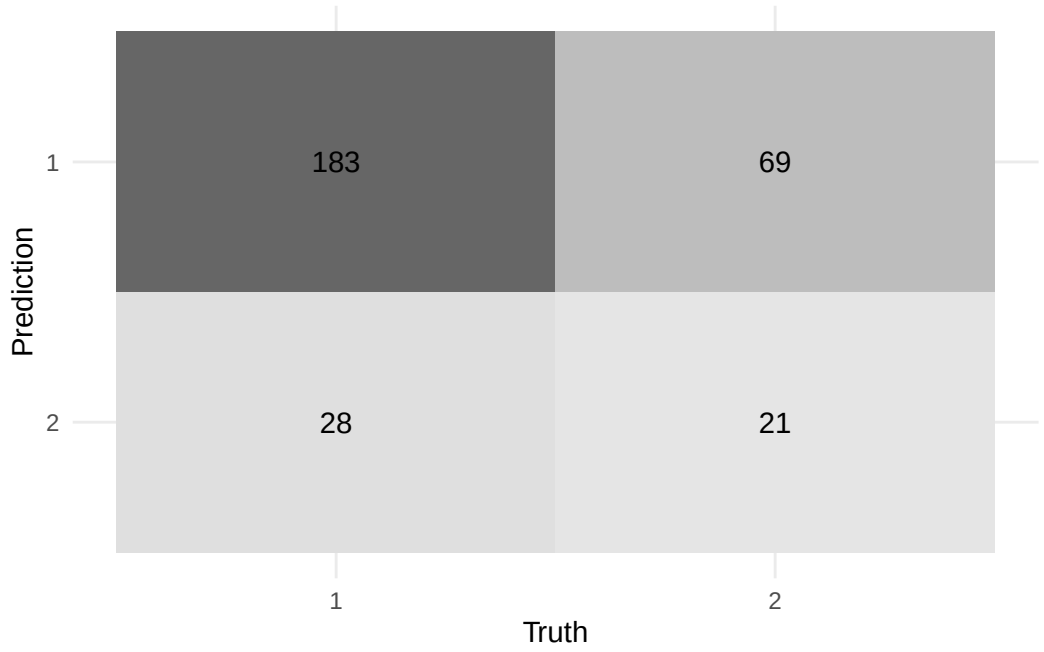


Figura 1.5: Matriz de confusión

1.5.2 Rendimiento

```
prest_treepred |>
  metrics(default, .pred_class)
```

```
# A tibble: 2 x 3
  .metric  .estimator .estimate
  <chr>    <chr>          <dbl>
1 accuracy binary         0.678
2 kap      binary         0.116
```

```
conf_metricas <- metric_set(accuracy, sensitivity, specificity)

conf_metricas(prest_treepred, truth = default, estimate = .pred_class, event_level = "seco
```

```
# A tibble: 3 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 accuracy    binary         0.678
2 sensitivity binary         0.233
3 specificity binary         0.867
```

1.6 Revisión de hiperparámetros "Cost Complexity" para la poda (pruning) del árbol

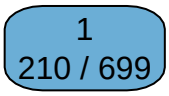
```
cp_tabla <- data.frame(tree_spec$fit$cptable)
```

```
best_cp <- cp_tabla$CP[which.min(cp_tabla$xerror)]
best_cp
```

```
[1] 0.02857143
```

```
pres_tree_poda <- decision_tree(cost_complexity = best_cp, mode = 'classification', engine
  fit(default~. , data = datos_entre)
```

```
rpart.plot(x = pres_tree_poda$fit, roundint = FALSE, extra = 3)
```



2 Bagging

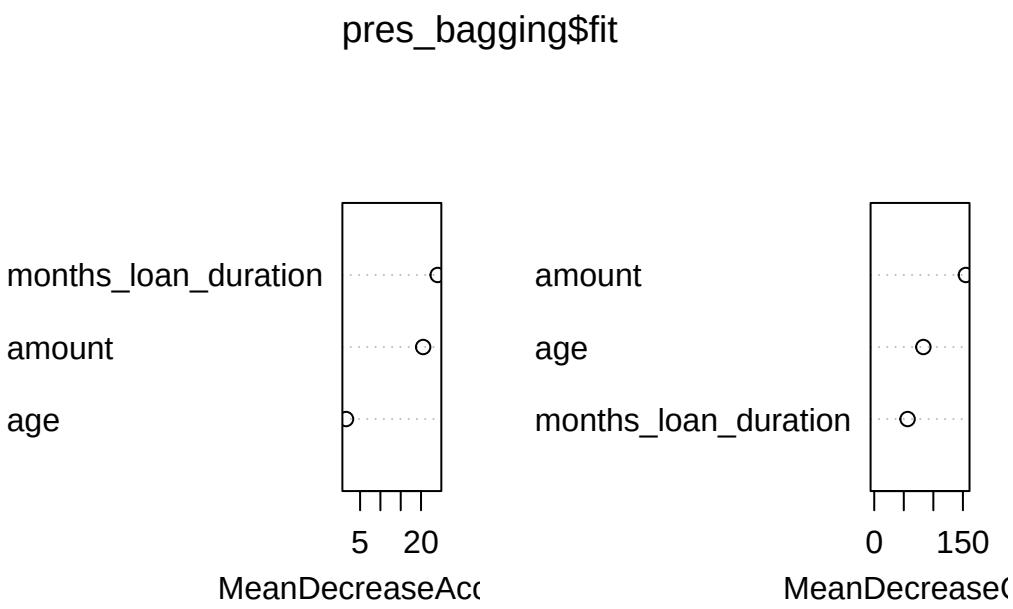
```
set.seed(100)

pres_bagging <- rand_forest() |>
  set_args(mtry = ncol(datos_entre)-1, trees = 500) |>
  set_engine("randomForest", importance = T) |>
  set_mode("classification") |>
  fit(default~., data = datos_entre)

pres_bagging
```

```
parsnip model object


Call:
 randomForest(x = maybe_data_frame(x), y = y, ntree = ~500, mtry = min_cols(~ncol(datos_en
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 3

        OOB estimate of  error rate: 33.91%
Confusion matrix:
    1  2 class.error
1 398 91   0.1860941
2 146 64   0.6952381
```

```
pres_bagging$fit$importance
```

```
                              1            2 MeanDecreaseAccuracy
age                 0.002097353  0.002506928          0.002148034
months_loan_duration 0.060221935 -0.015424791          0.037488705
amount              0.056667859 -0.018398649          0.034089155
                    MeanDecreaseGini
age                         83.09559
months_loan_duration        56.40868
amount                     154.98391
```

```r
varImpPlot(pres_bagging$fit)
```



2.1 Cálculo del rendimiento en testing

```r
pres_bagging_pred <- augment(pres_bagging,
                             new_data = datos_test,
                             type = "class")
pres_bagging_pred
```

```
# A tibble: 301 x 7
   .pred_class .pred_1 .pred_2   age months_loan_duration amount default
   <fct>         <dbl>   <dbl> <dbl>                <dbl>  <dbl> <fct>
 1 2             0.316   0.684    45                   42   7882 1
 2 1             0.808   0.192    53                   24   2835 1
 3 1             0.89    0.11     35                   36   6948 1
 4 1             0.692   0.308    28                   30   5234 2
 5 2             0.136   0.864    24                   48   4308 2
 6 1             0.768   0.232    53                   24   2424 1
 7 2             0.316   0.684    44                   24  12579 2
 8 1             0.942   0.058    26                   10   2069 1
 9 1             0.964   0.036    42                   12    409 1
10 2             0.352   0.648    63                   60   6836 2
# i 291 more rows
```

```r
conf_metricas(pres_bagging_pred, truth = default, estimate = .pred_class, event_level = "s
```

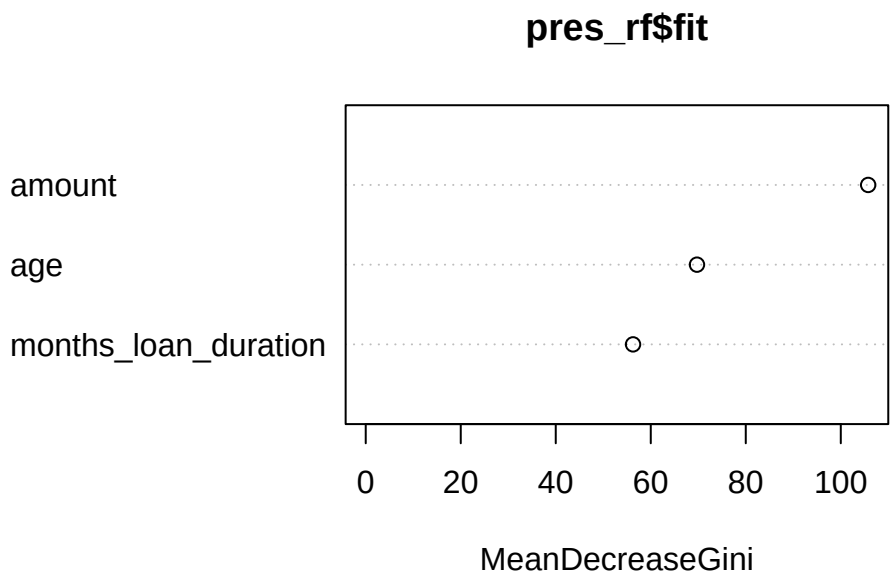
```
# A tibble: 3 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 accuracy    binary         0.678
2 sensitivity binary         0.367
3 specificity binary         0.810
```

3 Random Forest

```
set.seed(100)

pres_rf <- rand_forest() |>
  set_args(mtry = sqrt(ncol(datos_entre)-1),
           trees = 500) |>
  set_engine("randomForest") |>
  set_mode("classification") |>
  fit(default~., data=datos_entre)

pres_rf
```

```
parsnip model object


Call:
 randomForest(x = maybe_data_frame(x), y = y, ntree = ~500, mtry = min_cols(~sqrt(ncol(dat
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 1

        OOB estimate of  error rate: 32.19%
Confusion matrix:
    1  2 class.error
1 421 68   0.1390593
2 157 53   0.7476190
```

```r
varImpPlot(pres_rf$fit)
```




```r
pres_rfpred <- augment(pres_rf,
                       new_data = datos_test,
                      type = "class")
```

```r
conf_metricas(pres_rfpred, truth = default, estimate = .pred_class, event_level = "second"
```

```
# A tibble: 3 x 3
  .metric     .estimator .estimate
  <chr>       <chr>          <dbl>
1 accuracy    binary         0.694
2 sensitivity binary         0.267
3 specificity binary         0.877
```

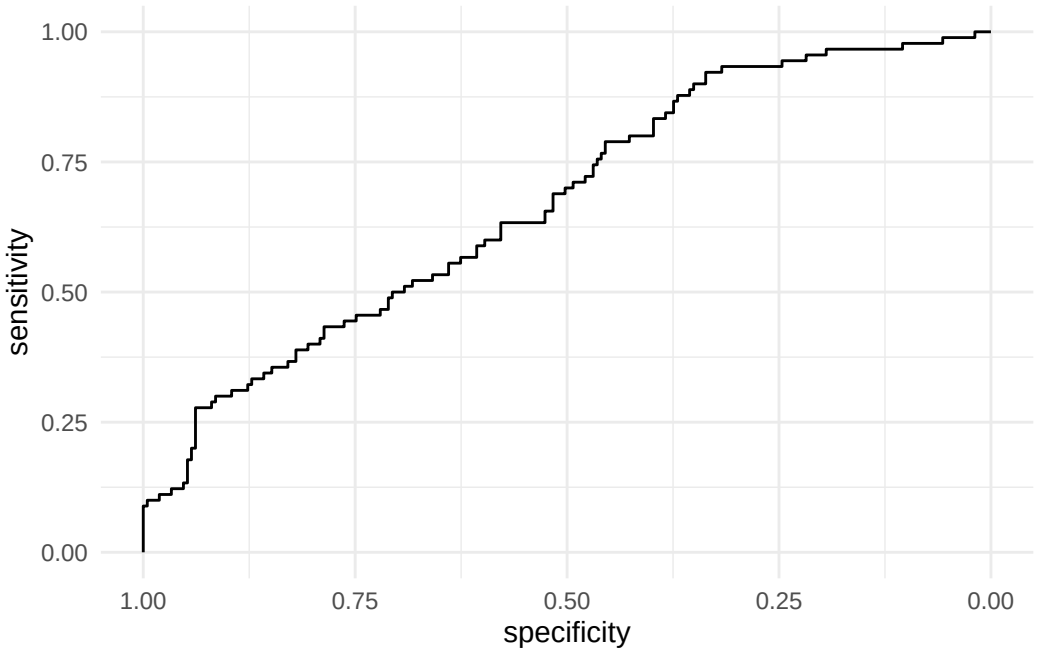
4 Comparación de modelos usando ROC

```
qdamod <- discrim_quad() %>%
  set_engine('MASS') %>%
  set_mode('classification') %>%
  fit(default ~ . , data = datos_entre)

pres_qda_pred <- augment(qdamod,
                         new_data = datos_test,
                         type = 'prob')
pres_qda_pred
```

```
# A tibble: 301 x 7
   .pred_class .pred_1 .pred_2   age months_loan_duration amount default
   <fct>         <dbl>   <dbl> <dbl>                <dbl>  <dbl> <fct>
 1 1           0.551     0.449    45                   42   7882 1
 2 1           0.823     0.177    53                   24   2835 1
 3 1           0.662     0.338    35                   36   6948 1
 4 1           0.747     0.253    28                   30   5234 2
 5 2           0.192     0.808    24                   48   4308 2
 6 1           0.813     0.187    53                   24   2424 1
 7 2           0.00158   0.998    44                   24  12579 2
 8 1           0.828     0.172    26                   10   2069 1
 9 1           0.853     0.147    42                   12    409 1
10 2           0.0782    0.922    63                   60   6836 2
# i 291 more rows
```

```
roc_qda <- roc(datos_test$default,   pres_qda_pred$.pred_2)
```

```
ggroc(roc_qda)
```




```
auc(roc_qda)
```

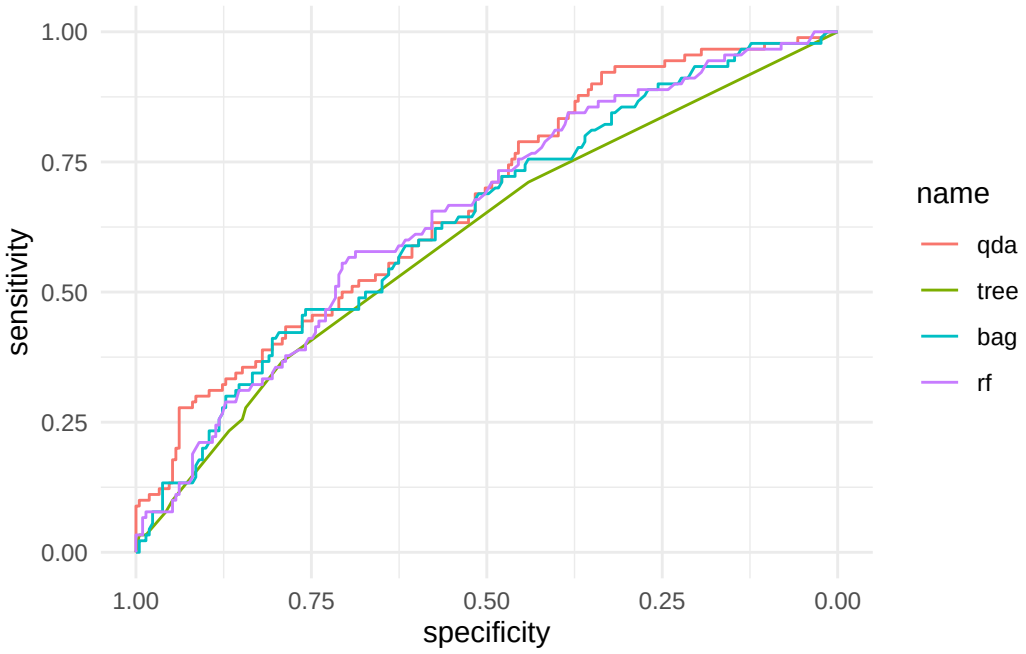
```
Area under the curve: 0.6702
```

```
roc_tree <- roc(datos_test$default,
                prest_treepred$.pred_2)
```

```
roc_bag <- roc(datos_test$default, pres_bagging_pred$.pred_2)
```

```
roc_rf <- roc(datos_test$default, pres_rfpred$.pred_2)
```

```
roc_curvas_unidas <- ggroc(list(qda = roc_qda,
            tree = roc_tree,
            bag = roc_bag,
            rf = roc_rf))
roc_curvas_unidas
```




```
auc(roc_qda)
```

```
Area under the curve: 0.6702
```

```
auc(roc_tree)
```

```
Area under the curve: 0.605
```

```
auc(roc_bag)
```

```
Area under the curve: 0.6392
```

```
auc(roc_rf)
```

```
Area under the curve: 0.6511
```

Referencias Bibliográficas

Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., & Dervieux, C. (2022). *Quarto* (1.2). https://doi.org/10.5281/zenodo.5960048

Feng, W. (2019). Learning apache spark with python. *Github*, 7.

Kabacoff, R. (2024). *Modern data visualization with r*. Chapman; Hall/CRC.

Rioux, J. (2022). *Data analysis with python and PySpark*. Manning Publications.

Wilke, C. O. (2019). *Fundamentals of data visualization: A primer on making informative and compelling figures*. O'Reilly Media.